

Punto 3.12 - Buenas prácticas en el acceso a datos y composición de consultas

Práctica

Ejercicio: Aplicar las buenas prácticas de acceso a datos a todos los métodos del repositorio de órdenes del proyecto AceriaData. Refactorizar los métodos existentes para usar proyecciones, AsNoTracking, AsSplitQuery, Any y FirstOrDefault. Crear un caso de uso que ejecute las consultas refactorizadas y analice el SQL generado.

Contexto del proyecto: En el punto 3.11 se profundizó en la composición de consultas y la ejecución diferida. En este punto se consolidan las buenas prácticas de acceso a datos, cerrando el Módulo 3. Estas buenas prácticas se usarán en el Módulo 4 para las optimizaciones de rendimiento y en el Módulo 5 para la persistencia empresarial.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Refactorizar los métodos del repositorio con buenas prácticas

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs para aplicar las buenas prácticas a los métodos existentes:

```
/// <summary>
/// Obtiene los resúmenes de las órdenes pendientes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

/// <summary>
/// Cuenta las órdenes pendientes.
/// Usa Any en lugar de Count cuando solo se quiere saber si hay elementos.
/// </summary>
public bool ExisteAlgunaOrdenPendiente()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Any(o => o.Estado == "Pendiente");
}

/// <summary>
/// Obtiene las órdenes con planchas y detalle usando AsSplitQuery.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// </summary>
```

```

public List<OrdenConPlanchasYDetalleDto> ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenConPlanchasYDetalleDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            Planchas = o.Planchas.Select(p => new PlanchaDto
            {
                Id = p.Id,
                Espesor = p.Espesor,
                Ancho = p.Ancho,
                Largo = p.Largo,
                Peso = p.Peso,
                Activa = p.Activa
            }).ToList(),
            Detalle = o.Detalle == null ? null : new DetalleDto
            {
                ComposicionQuimica = o.Detalle.ComposicionQuimica,
                TemperaturaColada = o.Detalle.TemperaturaColada,
                Notas = o.Detalle.Notas
            }
        })
        .ToList();
}

```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene los resúmenes de las órdenes pendientes. → descripción del método.

Línea 3: /// Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 4: /// Usa AsNoTracking porque es una consulta de solo lectura. → explicación del uso de AsNoTracking.

Línea 5: ///

→ cierre del comentario XML.

Línea 6: public List ObtenerResumenesPendientes() → declara el método.

Línea 8: return _context.OrdenesFabricacion → inicia la consulta.

Línea 9: .AsNoTracking() → aplica AsNoTracking.

Línea 10: .Where(o => o.Estado == "Pendiente") → filtra por estado.

Línea 11: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 12: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 13: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 14: Cliente = o.Cliente, → asigna el cliente.

Línea 15: Estado = o.Estado, → asigna el estado.

Línea 16: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 17: }) → cierra la proyección.

Línea 18: .ToList(); → materializa la consulta.

Línea 21: ///

→ inicio del comentario XML.

Línea 22: /// Cuenta las órdenes pendientes. → descripción del método.

Línea 23: /// Usa Any en lugar de Count cuando solo se quiere saber si hay elementos. → explicación del uso de Any.

Línea 24: ///

→ cierre del comentario XML.

Línea 25: public bool ExisteAlgunaOrdenPendiente() → declara el método.

Línea 27: return _context.OrdenesFabricacion → inicia la consulta.

Línea 28: .AsNoTracking() → aplica AsNoTracking.

Línea 29: .Any(o => o.Estado == "Pendiente"); → comprueba si hay órdenes pendientes.

Línea 32: ///

→ inicio del comentario XML.

Línea 33: /// Obtiene las órdenes con planchas y detalle usando AsSplitQuery. → descripción del método.

Línea 34: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 35: ///

→ cierre del comentario XML.

Línea 36: public List ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano() → declara el método.

Línea 38: return _context.OrdenesFabricacion → inicia la consulta.

Línea 39: .AsNoTracking() → aplica AsNoTracking.

Línea 40: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 41: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 42: .AsSplitQuery() → divide la consulta en varias.

Línea 43: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 44: .Select(o => new OrdenConPlanchasYDetalleDto → proyecta al DTO.

Línea 45: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 46: Cliente = o.Cliente, → asigna el cliente.

Línea 47: Estado = o.Estado, → asigna el estado.

Línea 48: Planchas = o.Planchas.Select(p => new PlanchaDto → proyecta las planchas.

Línea 49: Id = p.Id, → asigna el Id.

Línea 50: Espesor = p.Espesor, → asigna el espesor.

Línea 51: Ancho = p.Ancho, → asigna el ancho.

Línea 52: Largo = p.Largo, → asigna el largo.

Línea 53: Peso = p.Peso, → asigna el peso.

Línea 54: Activa = p.Activa → asigna el estado activo.

Línea 55: }).ToList(), → materializa la colección.

Línea 56: Detalle = o.Detalle == null ? null : new DetalleDto → comprueba si el detalle existe.

Línea 57: `ComposicionQuimica = o.Detalle.ComposicionQuimica`, → asigna la composición química.

Línea 58: `TemperaturaColada = o.Detalle.TemperaturaColada`, → asigna la temperatura.

Línea 59: `Notas = o.Detalle.Notas` → asigna las notas.

Línea 60: `}` → cierra la proyección del detalle.

Línea 61: `})` → cierra la proyección.

Línea 62: `.ToList();` → materializa la consulta.

Error común: si se aplica `AsNoTracking` a una consulta que devuelve entidades completas y después se modifican, los cambios no se guardan. Se debe usar `AsNoTracking` solo cuando no se van a modificar las entidades.

| Paso 3: Crear el caso de uso de buenas prácticas

Crear el archivo `src/AceriaData.Application/UseCases/BuenasPracticasUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class BuenasPracticasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public BuenasPracticasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");

        DemostrarProyeccionConAsNoTracking();
        DemostrarAnyEnLugarDeCount();
        DemostrarAsSplitQuerySinProductoCartesiano();
        DemostrarFirstOrDefault();
        DemostrarDocumentacionDeDecisiones();
    }

    private void DemostrarProyeccionConAsNoTracking()
    {
        Console.WriteLine("\n--- Proyección con AsNoTracking ---");

        var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientes();
        Console.WriteLine($"Resúmenes pendientes: {resúmenes.Count}");
        foreach (var resumen in resúmenes)
        {
            Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarAnyEnLugarDeCount()
    {
        Console.WriteLine("\n--- Any en lugar de Count ---");

        var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();
        Console.WriteLine($"Existe alguna orden pendiente: {existe}");
    }

    private void DemostrarAsSplitQuerySinProductoCartesiano()
    {
        Console.WriteLine("\n--- AsSplitQuery sin producto cartesiano ---");

        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();
        foreach (var orden in ordenes)
```

```

        {
            var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;
            Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle:
{detalle}");
        }
    }

    private void DemostrarFirstOrDefault()
    {
        Console.WriteLine("\n--- FirstOrDefault en lugar de First ---");

        var orden = _unidad.Ordenes.ObtenerPorNumeroOrden("OF-2024-9999");
        var resultado = orden is null ? "No encontrada" : orden.NumeroOrden;
        Console.WriteLine($"Orden OF-2024-9999: {resultado}");
    }

    private void DemostrarDocumentacionDeDecisiones()
    {
        Console.WriteLine("\n--- Documentación de decisiones ---");
        Console.WriteLine("Los métodos del repositorio incluyen comentarios XML que documentan las
decisiones de acceso a datos.");
        Console.WriteLine("Consultar el código fuente de OrdenRepositorio para más detalles.");
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class BuenasPracticasUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ==="); → muestra la cabecera.

Línea 18: DemostrarProyeccionConAsNoTracking(); → llama al método de proyección con AsNoTracking.

Línea 19: DemostrarAnyEnLugarDeCount(); → llama al método de Any en lugar de Count.

Línea 20: DemostrarAsSplitQuerySinProductoCartesiano(); → llama al método de AsSplitQuery.

Línea 21: DemostrarFirstOrDefault(); → llama al método de FirstOrDefault.

Línea 22: DemostrarDocumentacionDeDecisiones(); → llama al método de documentación.

Línea 25: private void DemostrarProyeccionConAsNoTracking() → declara el método.

Línea 27: Console.WriteLine("\n--- Proyección con AsNoTracking ---"); → muestra la cabecera.

Línea 29: var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientes(); → llama al método del repositorio.

Línea 30: Console.WriteLine(\$"Resúmenes pendientes: {resúmenes.Count}"); → muestra el número de resúmenes.

Línea 31: foreach (var resumen in resúmenes) → itera sobre los resúmenes.

Línea 33: Console.WriteLine(\$" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.Estado} | {resumen.FechaCreacion:dd/MM/yyyy}"); → muestra los datos.

Línea 36: private void DemostrarAnyEnLugarDeCount() → declara el método.

Línea 38: Console.WriteLine("\n--- Any en lugar de Count ---"); → muestra la cabecera.

Línea 40: `var existe = _unidad Ordenes.ExisteAlgunaOrdenPendiente();` → llama al método del repositorio.

Línea 41: `Console.WriteLine($"Existe alguna orden pendiente: {existe}");` → muestra el resultado.

Línea 44: `private void DemostrarAsSplitQuerySinProductoCartesiano()` → declara el método.

Línea 46: `Console.WriteLine("\n--- AsSplitQuery sin producto cartesiano ---");` → muestra la cabecera.

Línea 48: `var ordenes = _unidad Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();` → llama al método del repositorio.

Línea 49: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 51: `var detalle = orden.Detalle == null ? "Sin detalle" : orden.Detalle.ComposicionQuimica;` → comprueba si el detalle existe.

Línea 52: `Console.WriteLine($" {orden.NumeroOrden} | Planchas: {orden.Planchas.Count} | Detalle: {detalle}");` → muestra los datos.

Línea 56: `private void DemostrarFirstOrDefault()` → declara el método.

Línea 58: `Console.WriteLine("\n--- FirstOrDefault en lugar de First ---");` → muestra la cabecera.

Línea 60: `var orden = _unidad Ordenes.ObtenerPorNumeroOrden("OF-2024-9999");` → busca una orden que no existe.

Línea 61: `var resultado = orden is null ? "No encontrada" : orden.NumeroOrden;` → comprueba si la orden existe.

Línea 62: `Console.WriteLine($"Orden OF-2024-9999: {resultado}");` → muestra el resultado.

Línea 65: `private void DemostrarDocumentacionDeDecisiones()` → declara el método.

Línea 67: `Console.WriteLine("\n--- Documentación de decisiones ---");` → muestra la cabecera.

Línea 68: `Console.WriteLine("Los métodos del repositorio incluyen comentarios XML que documentan las decisiones de acceso a datos.");` → describe la documentación.

Línea 69: `Console.WriteLine("Consultar el código fuente de OrdenRepositorio para más detalles.");` → indica dónde consultar.

Error común: si no se documentan las decisiones, otros desarrolladores pueden modificarlas sin entender las consecuencias. Se deben documentar las decisiones de acceso a datos.

| Paso 4: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<BuenasPracticasUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 5: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 6: Insertar datos de prueba con planchas y detalle

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes, planchas y detalle:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
    context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);

    var detalle1 = new DetalleOrden { OrdenId = orden1.Id, ComposicionQuimica = "C: 0.45%, Mn: 0.75%", TemperaturaColada = 1550.5 };
    context.DetallesOrden.Add(detalle1);

    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base.

Línea 5: `context.Database.EnsureCreated();` → crea la base.

Línea 7: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden.

Línea 8: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden.

Línea 9: `var orden3 = new OrdenFabricacion { ... };` → crea la tercera orden.

Línea 10: `context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);` → registra las órdenes.

Línea 11: `context.SaveChanges();` → inserta las órdenes.

Línea 13: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha.

Línea 14: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha.

Línea 15: `var plancha3 = new PlanchaAcero { ... };` → crea la tercera plancha.

Línea 16: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);` → registra las planchas.

Línea 18: `var detalle1 = new DetalleOrden { ... }; → crea el detalle.`

Línea 19: `context.DetallesOrden.Add(detalle1); → registra el detalle.`

Línea 21: `context.SaveChanges(); → inserta las planchas y el detalle.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observan las proyecciones con `AsNoTracking`, el uso de `Any`, el uso de `AsSplitQuery` y el uso de `FirstOrDefault`.

Paso 8: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===

--- Proyección con AsNoTracking ---
Resúmenes pendientes: 2
  OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
  OF-2024-0002 | Constructora del Sur | Pendiente | 20/02/2024

--- Any en lugar de Count ---
Existe alguna orden pendiente: True

--- AsSplitQuery sin producto cartesiano ---
  OF-2024-0001 | Planchas: 2 | Detalle: C: 0.45%, Mn: 0.75%
  OF-2024-0002 | Planchas: 1 | Detalle: Sin detalle
  OF-2024-0003 | Planchas: 0 | Detalle: Sin detalle

--- FirstOrDefault en lugar de First ---
Orden OF-2024-9999: No encontrada

--- Documentación de decisiones ---
Los métodos del repositorio incluyen comentarios XML que documentan las decisiones de acceso a datos.
Consultar el código fuente de OrdenRepositorio para más detalles.
La primera sección muestra los resúmenes pendientes con proyección y AsNoTracking. La segunda sección
muestra el uso de Any. La tercera sección muestra el uso de AsSplitQuery sin producto cartesiano. La
cuarta sección muestra el uso de FirstOrDefault. La quinta sección muestra la documentación de
decisiones.

Observaciones: la proyección con AsNoTracking reduce el volumen de datos transferidos. El uso de Any es
más eficiente que Count. El uso de AsSplitQuery evita el producto cartesiano. El uso de FirstOrDefault
devuelve null en lugar de lanzar una excepción.

Paso 9: Diagnosticar un error común
Modificar el método ObtenerResumenesPendientes para no usar AsNoTracking:

csharp
public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}
```

```

    })
    .ToList();
}

```

Resultado esperado: el código funciona, pero las entidades se registran en el Change Tracker aunque no se vayan a modificar. Esto consume memoria innecesariamente.

Solución: añadir AsNoTracking antes del Where.

```

public List<OrdenResumenDto> ObtenerResumenesPendientes()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Resultado esperado con la solución: las entidades no se registran en el Change Tracker y se libera memoria.

Errores comunes del ejercicio

Error	Causa	Solución
AsNoTracking en consulta de escritura		Se aplicó AsNoTracking a una consulta que modifica datos
Count() > 0 en lugar de Any	Usar AsNoTracking solo en consultas de solo lectura	Se usó Count para saber si hay elementos
First en lugar de FirstOrDefault		Usar Any
FirstOrDefault		Se usó First cuando puede no haber resultados
Producto cartesiano		Usar
Materialización prematura	Se incluyeron varias colecciones sin AsSplitQuery	Usar AsSplitQuery
Materializar solo al final	Se llamó a ToList antes de aplicar todos los filtros	
N+1	Se accede a propiedades de navegación en un bucle sin Include	Usar Include
Falta de documentación	No se documentaron las decisiones	Añadir comentarios XML

Reto resuelto: Refactorizar un método con todas las buenas prácticas

Reto: Refactorizar el método ObtenerPorEstadoOrdenadosPorFecha del repositorio para aplicar todas las buenas prácticas: proyección, AsNoTracking, OrderBy, ToList al final y documentación XML. Añadir el método refactorizado, el DTO correspondiente y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```

csharp
List<OrdenResumenDto> ObtenerResumenesPorEstadoOrdenados(string estado);

```

Línea 1: List ObtenerResumenesPorEstadoOrdenados(string estado); → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```

/// <summary>
/// Obtiene los resúmenes de las órdenes de un estado concreto, ordenados por fecha.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// Materializa con ToList al final para ejecutar una sola consulta.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenesPorEstadoOrdenados(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.FechaCreacion)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene los resúmenes de las órdenes de un estado concreto, ordenados por fecha. → descripción del método.

Línea 3: /// Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 4: /// Usa AsNoTracking porque es una consulta de solo lectura. → explicación del uso de AsNoTracking.

Línea 5: /// Materializa con ToList al final para ejecutar una sola consulta. → explicación de la materialización al final.

Línea 6: ///

→ cierre del comentario XML.

Línea 7: public List ObtenerResumenesPorEstadoOrdenados(string estado) → declara el método.

Línea 9: return _context.OrdenesFabricacion → inicia la consulta.

Línea 10: .AsNoTracking() → aplica AsNoTracking.

Línea 11: .Where(o => o.Estado == estado) → filtra por estado.

Línea 12: .OrderBy(o => o.FechaCreacion) → ordena por fecha.

Línea 13: .Select(o => new OrdenResumenDto → proyecta al DTO.

Línea 14: NumeroOrden = o.NumeroOrden, → asigna el número.

Línea 15: Cliente = o.Cliente, → asigna el cliente.

Línea 16: Estado = o.Estado, → asigna el estado.

Línea 17: FechaCreacion = o.FechaCreacion → asigna la fecha.

Línea 18: }) → cierra la proyección.

Línea 19: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarResumenesPorEstadoOrdenados()
{
    Console.WriteLine("\n--- Resúmenes por estado ordenados ---");

    var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstadoOrdenados("Pendiente");
    Console.WriteLine($"Resúmenes pendientes: {resumenes.Count}");
    foreach (var resumen in resumenes)
    {
        Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.FechaCreacion:dd/MM/yyyy}");
    }
}
```

Línea 1: `private void DemostrarResumenesPorEstadoOrdenados()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Resúmenes por estado ordenados ---");` → muestra la cabecera.

Línea 5: `var resumenes = _unidad.Ordenes.ObtenerResumenesPorEstadoOrdenados("Pendiente");` → llama al método del repositorio.

Línea 6: `Console.WriteLine($"Resúmenes pendientes: {resumenes.Count}");` → muestra el número de resúmenes.

Línea 7: `foreach (var resumen in resumenes)` → itera sobre los resúmenes.

Línea 9: `Console.WriteLine($" {resumen.NumeroOrden} | {resumen.Cliente} | {resumen.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarResumenesPorEstadoOrdenados();
```

Paso 5: Ejecutar `dotnet run` y verificar que los resúmenes se muestran ordenados por fecha.

Resultado esperado: los resúmenes de las órdenes pendientes se muestran ordenados por fecha.

Analogía final

Las buenas prácticas de acceso a datos en una acería son como las normas de seguridad y eficiencia de la planta. No se trata solo de producir acero, sino de producirlo de forma segura, eficiente y sostenible. Usar proyecciones es como pedir solo los datos que se necesitan: no se pide la carpeta completa si solo se necesita el número de orden. Usar `AsNoTracking` es como no registrar cada plancha en el libro de producción si solo se va a consultar. Usar `Any` es como comprobar si hay planchas en el almacén sin contarlas todas. Usar `FirstOrDefault` es como buscar una plancha por su número y devolver null si no existe, en lugar de lanzar una alarma. Usar `AsSplitQuery` es como dividir una orden de búsqueda grande en varias más pequeñas para evitar un envío masivo. Documentar las decisiones es como escribir las normas en el manual de la planta para que todos las conozcan. Así funcionan las buenas prácticas en EF Core: producen código más eficiente, más mantenible y más predecible.

Resultado esperado

Al final del ejercicio, deberías haber:

Refactorizado los métodos del repositorio con buenas prácticas.

Añadido `AsNoTracking` a las consultas de solo lectura.

Usado `Any` en lugar de `Count`.

Usado `AsSplitQuery` cuando se incluyen varias colecciones.

Usado `FirstOrDefault` en lugar de `First`.

Documentado las decisiones de acceso a datos con comentarios XML.

Creado el caso de uso BuenasPracticasUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con planchas y detalle.

Ejecutado las demostraciones de proyección con AsNoTracking, Any, AsSplitQuery y FirstOrDefault.

Diagnosticado el error de no usar AsNoTracking.

Creado el método ObtenerResumenesPorEstadoOrdenados.

Resumen del estado del proyecto AceriaData al final del Módulo 3 Al final del Módulo 3, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infrastructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IOrdenRepositorio y IUnidadDeTrabajo, los casos de uso CrearOrdenUseCase, ListarOrdenesUseCase, ConsultasLinqUseCase, ConsultasBasicasUseCase, ProyeccionesUseCase, ProyeccionesDtoUseCase, AgregacionesUseCase, AgrupacionesUseCase, JoinsUseCase, CargaEagerUseCase, CargaLazyUseCase, CargaExplicitUseCase, ComposicionConsultasUseCase y BuenasPracticasUseCase, y los DTOs.

La infraestructura con el AceriaDbContext, las configuraciones de Fluent API, las implementaciones de repositorios y la unidad de trabajo.

La consola con el método Main y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El Soft Delete implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.
